

Domain 2: Reliability and Business Continuity

AWS Certified SysOps Administrator – Associate (SOA-C03)

Exam Weight: ~16% of scored content

Source: AWS Certified SysOps Administrator – Associate Exam Guide

Overview

Domain 2 covers the strategies and AWS services used to build systems that remain available, recover quickly from failures, and scale to meet demand. A CloudOps engineer must understand how to configure Auto Scaling, implement caching, manage database scaling, design highly available architectures with load balancers and Route 53, and implement comprehensive backup and disaster recovery strategies.

The three core tasks are:

1. **Task 2.1** – Implement scalability and elasticity
 2. **Task 2.2** – Implement highly available and resilient environments
 3. **Task 2.3** – Implement backup and restore strategies
-

Task 2.1: Implement Scalability and Elasticity

Scalability is the ability of a system to handle increased load by adding resources. Elasticity extends this by also releasing resources when demand decreases, optimizing cost. AWS provides native scaling mechanisms for compute, caching, and managed databases.

Skill 2.1.1 – Configure and Manage Scaling Mechanisms in Compute Environments

Amazon EC2 Auto Scaling

Amazon EC2 Auto Scaling automatically adjusts the number of EC2 instances in a group to maintain performance and minimize cost. Instances are organized into **Auto Scaling groups (ASGs)**, which define the minimum, maximum, and desired capacity.

Source: What is Amazon EC2 Auto Scaling? – Amazon EC2 Auto Scaling

Auto Scaling Group Key Properties

Property	Description
Minimum capacity	The floor — ASG will never go below this count.
Maximum capacity	The ceiling — ASG will never exceed this count.
Desired capacity	The target number of instances at any given time.
Launch template	Defines the AMI, instance type, key pair, security groups, and other configuration for new instances.
Availability Zones	ASGs distribute instances evenly across specified AZs for high availability.
Health checks	EC2 status checks or ELB health checks determine whether an instance is healthy. Unhealthy instances are replaced automatically.

Scaling Policy Types

Policy Type	How It Works	Best For
Target tracking	Maintains a metric at a target value (e.g., keep CPU at 50%). AWS manages the CloudWatch alarms automatically.	Most use cases — simplest to configure.
Step scaling	Adjusts capacity in steps based on how far a metric deviates from a threshold. Allows different step sizes for different alarm breach magnitudes.	Workloads needing fine-grained control.
Simple scaling	Adds or removes a fixed number of instances when a single CloudWatch alarm fires. Includes a cooldown period before the next action.	Legacy; target tracking is preferred.
Scheduled scaling	Scales capacity at a specific time using cron expressions.	Predictable load patterns (e.g., business hours).
Predictive scaling	Uses machine learning to forecast future load and proactively adjusts capacity.	Cyclical workloads with recurring patterns.

Cooldown Periods

A **cooldown period** prevents Auto Scaling from launching or terminating additional instances before the effects of a previous scaling activity are visible. The default cooldown is 300 seconds. For scale-in policies, a shorter cooldown can reduce costs by terminating instances faster after load drops.

Source: Scaling cooldowns for Amazon EC2 Auto Scaling

Instance Warmup

For target tracking and step scaling, you can define an **instance warmup time** — the period after a new instance launches before its metrics contribute to the scaling metric. This prevents premature scale-out triggered by a new instance that hasn't yet reached steady-state load.

Lifecycle Hooks

Lifecycle hooks pause instance launch or termination to allow custom actions (e.g., installing software, draining connections). The instance remains in a `Pending:Wait` or `Terminating:Wait` state until the hook completes or times out.

Mixed Instance Policies and Spot Instances

ASGs support **mixed instance policies** that combine On-Demand and Spot Instances within a single group. This reduces cost while maintaining availability. **Capacity Rebalancing** proactively replaces Spot Instances at elevated interruption risk before they are reclaimed.

Application Auto Scaling

Beyond EC2, **Application Auto Scaling** provides scaling for other AWS services using the same target tracking and step scaling policies:

- Amazon ECS services (task count)
 - Amazon DynamoDB tables and global secondary indexes
 - Amazon Aurora replicas
 - AWS Lambda provisioned concurrency
 - Amazon SageMaker endpoint variants
-

AWS Auto Scaling (Scaling Plans)

AWS Auto Scaling (the unified service) lets you create **scaling plans** that coordinate scaling across multiple resource types for a single application. It supports predictive scaling and dynamic scaling together, and integrates with AWS CloudFormation stacks and tags to discover resources automatically.

Skill 2.1.2 – Implement Caching Using AWS Services (CloudFront, ElastiCache)

Caching reduces latency and offloads backend systems by serving frequently accessed data from a fast, in-memory or edge layer. AWS provides two primary caching services: **Amazon CloudFront** for content delivery and **Amazon ElastiCache** for application-layer caching.

Amazon CloudFront

Amazon CloudFront is a globally distributed content delivery network (CDN) that caches content at **edge locations** close to end users. It reduces latency for static assets, dynamic content, APIs, and streaming media.

Source: *What is Amazon CloudFront? – Amazon CloudFront*

Key CloudFront Concepts

Concept	Description
Distribution	The CloudFront configuration that maps a domain name to one or more origins.
Origin	The source of the content — an S3 bucket, ALB, EC2 instance, or any HTTP server.
Edge location	A data center in the CloudFront network where content is cached.
Regional edge cache	A larger cache between edge locations and the origin, reducing origin load.
Cache behavior	Rules that define how CloudFront handles requests for specific URL path patterns.
TTL (Time to Live)	How long an object stays in the cache before CloudFront checks the origin for a newer version.
Cache invalidation	Removes objects from the cache before their TTL expires. Useful after deployments.
Origin Shield	An additional caching layer that reduces the number of requests that reach the origin.

CloudFront and Scalability

CloudFront absorbs traffic spikes at the edge, preventing them from reaching origin servers. Combined with S3 static hosting or an ALB origin, it enables near-unlimited read scalability for web applications.

Amazon ElastiCache

Amazon ElastiCache is a fully managed, in-memory caching service that supports two engines: **Redis** (now called ElastiCache for Redis OSS or Valkey) and **Memcached**. It reduces database load by caching query results, session data, and computed values.

Source: *Caching strategies – Amazon ElastiCache*

ElastiCache Engines Compared

Feature	Redis (OSS / Valkey)	Memcached
Data structures	Strings, hashes, lists, sets, sorted sets, bitmaps, streams	Strings only
Persistence	Optional (RDB snapshots, AOF)	None
Replication	Yes (primary + replicas)	No
Cluster mode	Yes (sharding across nodes)	Yes (multi-node)
Pub/Sub	Yes	No
Transactions	Yes	No
Use cases	Sessions, leaderboards, queues, pub/sub	Simple key-value caching

Caching Strategies

Strategy	Description	Pros	Cons
Lazy loading	Data is loaded into cache only when requested (cache miss triggers a DB read).	Cache only contains requested data.	First request is slow (cache miss penalty). Stale data possible.
Write-through	Data is written to cache and DB simultaneously on every write.	Cache is always up to date.	Write latency increases. Cache may hold data never read.
TTL (Time to Live)	Each cached key has an expiration time. After expiry, the next read fetches fresh data.	Limits stale data.	Requires tuning per use case.

ElastiCache Cluster Modes

- **Cluster mode disabled (Redis):** Single shard with one primary and up to 5 read replicas. Supports vertical scaling (changing node type) and horizontal read scaling.
- **Cluster mode enabled (Redis):** Data is partitioned across multiple shards (up to 500). Enables horizontal write scaling. Supports online resharding without downtime.

Skill 2.1.3 – Configure and Manage Scaling in AWS Managed Databases (RDS, DynamoDB)

Amazon RDS Scaling

Amazon RDS supports both vertical and horizontal scaling strategies.

Source: Scaling and high availability in Amazon RDS

Vertical Scaling (Instance Resizing)

You can change the DB instance class (e.g., from `db.t3.medium` to `db.r6g.large`) to increase CPU, memory, and network throughput. The change can be applied immediately or during the next maintenance window. For Multi-AZ deployments, RDS performs the resize with minimal downtime by failing over to the standby.

Horizontal Scaling with Read Replicas

Read replicas offload read traffic from the primary instance using asynchronous replication. Key points:

- Supported engines: MySQL, MariaDB, PostgreSQL, Oracle, SQL Server, Aurora.
- Up to **15 read replicas** for Aurora; up to **5** for other engines.
- Read replicas can be in the same Region or a different Region (cross-Region read replicas).
- A read replica can be promoted to a standalone DB instance for disaster recovery.
- Read replicas can themselves be Multi-AZ for additional durability.

RDS Storage Auto Scaling

RDS Storage Auto Scaling automatically increases storage capacity when free space falls below a threshold. You set a **maximum storage threshold**, and RDS scales storage in increments without downtime.

Amazon RDS Proxy

RDS Proxy is a fully managed database proxy that pools and shares database connections. It reduces the overhead of opening new connections (especially from Lambda functions) and improves resilience by automatically routing traffic to a new primary during failover.

Amazon Aurora Scaling

Amazon Aurora extends RDS with additional scaling capabilities:

- **Aurora Auto Scaling:** Automatically adds or removes Aurora Replicas based on CloudWatch metrics (e.g., `AuroraDatabaseConnections`, `CPUUtilization`). Uses Application Auto Scaling under the hood.
 - **Aurora Serverless v2:** Scales compute capacity in fine-grained increments (Aurora Capacity Units, or ACUs) in response to application demand, with near-instant scaling.
 - **Aurora Global Database:** Spans multiple AWS Regions with a primary Region for writes and up to 5 secondary Regions for low-latency reads and disaster recovery.
-

Amazon DynamoDB Scaling

DynamoDB offers two capacity modes that determine how throughput is provisioned and scaled.

Capacity Modes

Mode	Description	Best For
On-demand	DynamoDB automatically scales to handle any request volume. You pay per read/write request unit. No capacity planning required.	Unpredictable or spiky workloads, new applications.
Provisioned	You specify read capacity units (RCUs) and write capacity units (WCUs). Can be combined with DynamoDB Auto Scaling.	Predictable workloads where cost optimization matters.

DynamoDB Auto Scaling (Provisioned Mode)

DynamoDB Auto Scaling uses **Application Auto Scaling** and CloudWatch alarms to automatically adjust provisioned RCUs and WCUs. You define:

- **Minimum and maximum capacity** for the table or GSI.
- **Target utilization** (e.g., 70% of provisioned capacity consumed).

When consumed capacity exceeds the target, Auto Scaling increases provisioned capacity. When it drops below, capacity is reduced.

DynamoDB Global Tables

Global Tables provide multi-Region, multi-active replication. Each Region can accept reads and writes, and changes are replicated asynchronously. This supports both global scalability and disaster recovery.

Task 2.2: Implement Highly Available and Resilient Environments

High availability (HA) means a system continues to operate even when individual components fail. On AWS, HA is achieved by distributing workloads across multiple Availability Zones (AZs) and using services that automatically detect and route around failures.

Skill 2.2.1 – Configure and Troubleshoot Elastic Load Balancing (ELB) and Amazon Route 53 Health Checks

Elastic Load Balancing (ELB)

Elastic Load Balancing automatically distributes incoming application traffic across multiple targets (EC2 instances, containers, Lambda functions, IP addresses) in one or more Availability Zones. ELB continuously monitors the health of registered targets and routes traffic only to healthy ones.

ELB Types

Load Balancer	Layer	Protocols	Key Features
Application Load Balancer (ALB)	Layer 7 (HTTP/HTTPS)	HTTP, HTTPS, gRPC, WebSocket	Content-based routing (path, host, headers, query strings), Lambda targets, WAF integration, sticky sessions.
Network Load Balancer (NLB)	Layer 4 (TCP/UDP)	TCP, UDP, TLS	Ultra-low latency, static IP per AZ, Elastic IP support, preserves source IP, handles millions of requests/second.
Gateway Load Balancer (GWLB)	Layer 3 (IP)	IP	Deploys, scales, and manages third-party virtual appliances (firewalls, IDS/IPS). Uses GENEVE protocol.
Classic Load Balancer (CLB)	Layer 4 & 7	HTTP, HTTPS, TCP, SSL	Legacy; not recommended for new deployments.

ALB Routing Rules

ALB listener rules evaluate conditions in order and forward traffic to a target group based on:

- **Path-based routing:** `/api/*` → API target group, `/static/*` → S3 or static server target group.
- **Host-based routing:** `api.example.com` → API target group, `www.example.com` → web target group.
- **Header-based routing:** Route based on HTTP headers (e.g., `User-Agent`, custom headers).
- **Query string routing:** Route based on URL query parameters.
- **IP-based routing:** Route based on source IP CIDR.

ELB Health Checks

Health checks determine whether a target is healthy enough to receive traffic. Each target group has its own health check configuration.

Setting	Description
Protocol	HTTP, HTTPS, or TCP (for NLB).
Path	The URL path to check (e.g., <code>/health</code>). HTTP/HTTPS only.
Healthy threshold	Number of consecutive successful checks before a target is considered healthy.
Unhealthy threshold	Number of consecutive failed checks before a target is considered unhealthy.
Timeout	Time to wait for a response before marking the check as failed.
Interval	Time between health checks.

Troubleshooting ELB Health Checks

Common reasons targets fail health checks:

Issue	Cause	Resolution
HTTP 4xx/5xx from target	Application error or misconfigured health check path.	Verify the health check path returns HTTP 200. Check application logs.
Connection timeout	Security group blocks ELB health check traffic.	Allow inbound traffic from the ELB security group (ALB) or from the VPC CIDR (NLB) on the health check port.
Target in wrong state	Instance not fully started or in a stopped state.	Check EC2 instance status. Review Auto Scaling lifecycle hooks.
Incorrect port	Health check port doesn't match the application port.	Verify the target group port and health check port settings.

ELB Access Logs

ELB can deliver access logs to an S3 bucket. Logs capture request details including timestamp, client IP, request path, response code, and latency. Useful for troubleshooting and auditing.

ELB Cross-Zone Load Balancing

- **ALB:** Cross-zone load balancing is always enabled. Traffic is distributed evenly across all registered targets in all enabled AZs.
- **NLB:** Cross-zone load balancing is disabled by default. When disabled, each NLB node distributes traffic only to targets in its own AZ.

Amazon Route 53 Health Checks

Amazon Route 53 health checks monitor the health of endpoints (web servers, email servers, or other resources) and can be used to route traffic away from unhealthy endpoints.

Source: How Amazon Route 53 checks the health of your resources

Health Check Types

Type	Description
Endpoint health check	Monitors an IP address or domain name. Supports HTTP, HTTPS, and TCP. Route 53 health checkers send requests from multiple locations globally.
Calculated health check	Combines the results of multiple health checks using AND/OR logic. Useful for monitoring a group of resources.
CloudWatch alarm health check	Reports the state of a CloudWatch alarm. Allows monitoring of any metric (e.g., DynamoDB throttles, SQS queue depth).

Health Check Configuration Options

- **Request interval:** 30 seconds (standard) or 10 seconds (fast, higher cost).
- **Failure threshold:** Number of consecutive failures before the endpoint is considered unhealthy (1–10).
- **String matching:** Route 53 can check that the response body contains a specific string (first 5,120 bytes).
- **Latency graphs:** Route 53 can measure and graph the latency of health check requests.
- **SNS notifications:** Route 53 can send notifications via Amazon SNS when health check status changes.

Route 53 Routing Policies for High Availability

Routing Policy	Description
Failover	Routes traffic to a primary resource; fails over to a secondary if the primary health check fails.
Weighted	Distributes traffic across multiple resources in specified proportions. Set weight to 0 to stop sending traffic to a resource.
Latency-based	Routes traffic to the AWS Region with the lowest latency for the user.
Geolocation	Routes traffic based on the geographic location of the user.
Geoproximity	Routes traffic based on geographic location with optional bias to shift traffic toward or away from a resource.
Multivalue answer	Returns up to 8 healthy records in response to DNS queries. Provides basic load balancing at the DNS level.

Combining ELB and Route 53

A common HA pattern:

1. Deploy an ALB in each Region (or AZ).
2. Create Route 53 health checks pointing to each ALB.
3. Use a **failover routing policy** to route traffic to the primary Region's ALB, with automatic failover to the secondary Region if the primary health check fails.

Skill 2.2.2 – Configure Fault-Tolerant Systems (Multi-AZ Deployments)

What is Multi-AZ?

AWS Regions are divided into **Availability Zones (AZs)** — physically separate data centers with independent power, cooling, and networking. Deploying resources across multiple AZs protects against the failure of a single data center.

Multi-AZ is both a general architectural principle and a specific feature of several AWS managed services.

Multi-AZ for Amazon RDS

When you enable Multi-AZ for an RDS DB instance, AWS automatically provisions a **synchronous standby replica** in a different AZ. The standby is not accessible for reads — it exists solely for failover.

Source: Multi-AZ DB instance deployments – Amazon RDS

How RDS Multi-AZ Failover Works

1. RDS detects a failure (primary instance failure, AZ disruption, or maintenance).
2. RDS updates the DNS record for the DB endpoint to point to the standby.
3. The standby is promoted to primary.
4. A new standby is provisioned in another AZ.

Failover typically completes in **60–120 seconds**. Applications must use the RDS endpoint DNS name (not the IP address) to benefit from automatic failover.

Multi-AZ DB Cluster (Three-Node)

RDS also supports a **Multi-AZ DB cluster** with one writer and two readable standby instances across three AZs. This provides:

- Faster failover (typically under 35 seconds).
- Read scaling from the two standby instances.
- Supported for MySQL and PostgreSQL.

Multi-AZ for Amazon Aurora

Aurora's architecture is inherently multi-AZ. The **Aurora storage layer** replicates data across 3 AZs with 6 copies of data (2 copies per AZ). Aurora Replicas can be placed in different AZs and serve as failover targets.

- **Failover priority:** Each Aurora Replica has a tier (0–15). The replica with the lowest tier number is promoted first during failover.
 - **Failover time:** Typically under 30 seconds for Aurora.
-

Multi-AZ for Other Services

Service	Multi-AZ Behavior
Amazon ElastiCache (Redis)	Enable Multi-AZ with automatic failover. If the primary node fails, a read replica in another AZ is promoted automatically.
Amazon EFS	Stores data redundantly across multiple AZs by default (Standard storage class).
Amazon S3	Stores objects redundantly across multiple AZs within a Region by default (Standard storage class).
Amazon DynamoDB	Data is automatically replicated across 3 AZs within a Region.
Amazon SQS	Messages are stored redundantly across multiple AZs.

EC2 High Availability Patterns

For EC2-based workloads, Multi-AZ HA is achieved by:

- 1. Auto Scaling groups spanning multiple AZs:** ASG distributes instances evenly across AZs. If an AZ becomes unavailable, ASG launches replacement instances in the remaining AZs.
- 2. Elastic Load Balancing:** Routes traffic only to healthy instances across AZs.
- 3. Elastic IP with failover scripts:** For single-instance workloads, an Elastic IP can be remapped to a standby instance in another AZ using a Lambda function or Systems Manager Automation runbook triggered by a CloudWatch alarm.

Fault Tolerance vs. High Availability

Concept	Definition	Example
High Availability	System remains operational despite component failures, possibly with brief interruption.	RDS Multi-AZ with 60-second failover.
Fault Tolerance	System continues operating without interruption even when components fail.	S3 (no single point of failure; requests are served even during AZ failures).
Disaster Recovery	System can be restored after a large-scale failure (Region-level).	Cross-Region RDS read replica promoted after Region failure.

Task 2.3: Implement Backup and Restore Strategies

A robust backup and restore strategy ensures that data can be recovered after accidental deletion, corruption, ransomware, or a large-scale disaster. AWS provides both native per-service backup capabilities and a centralized service — **AWS Backup** — for managing backups across multiple services.

Skill 2.3.1 – Automate Snapshots and Backups Using AWS Services (AWS Backup)

AWS Backup

AWS Backup is a fully managed, policy-based service that centralizes and automates data protection across AWS services. It removes the need for custom scripts and per-service backup configurations.

Source: What is AWS Backup? – AWS Backup

Supported Resources

AWS Backup supports backups for:

Service	Resource Type
Amazon EC2	Instances (AMI + EBS snapshots)
Amazon EBS	Volumes (snapshots)
Amazon RDS	DB instances and clusters (snapshots)
Amazon Aurora	DB clusters (snapshots)
Amazon DynamoDB	Tables (on-demand and continuous backups)
Amazon EFS	File systems
Amazon FSx	Windows File Server, Lustre, NetApp ONTAP, OpenZFS
Amazon S3	Buckets (continuous backups)
AWS Storage Gateway	Volumes
Amazon DocumentDB	Clusters
Amazon Neptune	Clusters
VMware workloads	On-premises VMs (via AWS Backup gateway)

Backup Plans

A **backup plan** defines when and how resources are backed up. Each plan contains one or more **backup rules**, each specifying:

- **Schedule:** A cron expression or rate (e.g., every 12 hours, daily at 5:00 AM UTC).
- **Backup window:** The time window within which the backup must start.
- **Retention period:** How long to keep the backup (days, weeks, months, years, or indefinitely).
- **Lifecycle:** Optionally transition backups to cold storage after a specified number of days to reduce cost.
- **Copy to another Region:** Automatically copy backups to a different AWS Region for disaster recovery.
- **Copy to another account:** Copy backups to a different AWS account for additional isolation.

Assigning Resources to Backup Plans

Resources can be assigned to backup plans by:

- **Resource ID:** Explicitly specify individual resources.
- **Tags:** Apply a backup plan to all resources with a specific tag (e.g., `Backup=Daily`). This is the recommended approach for scalability.

Backup Vaults

Backups are stored in **backup vaults** — logical containers that organize and secure backups. Key features:

- **Encryption:** Each vault is encrypted with a KMS key. Backups inherit the vault's encryption.
- **Access policies:** Resource-based policies control who can access, restore, or delete backups in a vault.
- **Vault Lock (WORM):** Enables Write Once Read Many (WORM) protection. Once enabled, backups in the vault cannot be deleted before their retention period expires — even by the root account. Supports compliance requirements (e.g., SEC Rule 17a-4).

AWS Backup Audit Manager

AWS Backup Audit Manager provides pre-built and custom compliance controls to audit backup activity. It generates compliance reports showing whether resources are backed up according to policy, and integrates with AWS Config for continuous compliance monitoring.

Per-Service Backup Capabilities

Amazon EC2 – AMIs and EBS Snapshots

- **Amazon Machine Image (AMI):** A complete backup of an EC2 instance, including the root volume and any additional EBS volumes. Used to launch new instances with the same configuration.
- **EBS Snapshot:** A point-in-time backup of an EBS volume stored in S3 (managed by AWS). Snapshots are incremental — only changed blocks are stored after the first snapshot.
- **Amazon Data Lifecycle Manager (DLM):** Automates the creation, retention, and deletion of EBS snapshots and EBS-backed AMIs using lifecycle policies.

Source: Amazon EC2 backup and recovery with snapshots and AMIs – AWS Prescriptive Guidance

Amazon RDS – Automated Backups and Snapshots

- **Automated backups:** RDS automatically backs up the DB instance daily during the backup window and retains transaction logs to support point-in-time recovery (PITR). Retention period: 0–35 days (0 disables automated backups).
- **Manual snapshots:** User-initiated snapshots that persist until explicitly deleted.

- **Cross-Region backup replication:** RDS can replicate automated backups to another Region for disaster recovery.

Source: Introduction to backups – Amazon RDS

Amazon DynamoDB – On-Demand and Continuous Backups

- **On-demand backups:** Full table backups created manually or via AWS Backup. No impact on table performance.
- **Point-in-time recovery (PITR):** Continuous backups that allow restoration to any second within the last 35 days. Must be explicitly enabled per table.

Amazon S3 – Replication and Versioning

- **S3 Versioning:** Preserves every version of every object. Protects against accidental deletion and overwrites. When an object is deleted, S3 inserts a delete marker rather than permanently removing the object.
- **S3 Cross-Region Replication (CRR):** Asynchronously replicates objects to a bucket in another Region. Useful for DR and compliance.
- **S3 Same-Region Replication (SRR):** Replicates objects within the same Region (e.g., to a different account).

Skill 2.3.2 – Use Various Methods to Restore Databases (Point-in-Time Restore) to Meet RTO, RPO, and Cost Requirements

Recovery Objectives

Metric	Definition	Impact
RTO (Recovery Time Objective)	Maximum acceptable time to restore service after a failure.	Drives the choice of DR strategy and infrastructure. Lower RTO = higher cost.
RPO (Recovery Point Objective)	Maximum acceptable amount of data loss measured in time.	Drives backup frequency. Lower RPO = more frequent backups = higher cost.

RDS Point-in-Time Restore (PITR)

RDS PITR allows you to restore a DB instance to any second within the automated backup retention period (up to 35 days). The restore creates a **new DB instance** — it does not overwrite the existing one.

How it works:

1. RDS restores the most recent daily snapshot.
2. RDS replays transaction logs up to the specified point in time.
3. A new DB instance is created with the restored data.

Use cases:

- Recovering from accidental data deletion or corruption.
- Recovering from a failed schema migration.

Considerations:

- The new instance has a different endpoint — applications must be updated to point to it.
 - PITR is not available for read replicas.
 - Restoring to a point before the oldest automated backup is not possible.
-

Aurora Point-in-Time Restore

Aurora PITR works similarly to RDS PITR but leverages Aurora's continuous backup to S3. Aurora can restore to any second within the backup retention period (1–35 days). Aurora Backtrack (MySQL-compatible only) provides a faster alternative — it rewinds the database in place without creating a new cluster, with a backtrack window of up to 72 hours.

DynamoDB Point-in-Time Recovery

When PITR is enabled on a DynamoDB table, you can restore the table to any second within the last 35 days. The restore creates a **new table** — the original table is not affected. Restore time depends on table size.

Choosing a Restore Method Based on RTO/RPO

Scenario	Recommended Method	Typical RTO	Typical RPO
Accidental row deletion in RDS	PITR to just before the deletion	Minutes to hours (new instance creation)	Seconds
Full RDS instance failure	Restore from automated backup or snapshot	Minutes to hours	Hours (last backup)
DynamoDB table corruption	PITR restore to new table	Minutes to hours	Seconds (if PITR enabled)
EC2 instance failure	Launch from AMI or restore EBS snapshot	Minutes	Hours (last snapshot)
Region-level disaster	Promote cross-Region read replica or restore from cross-Region backup	Hours	Minutes to hours

Skill 2.3.3 – Implement Versioning for Storage Services (Amazon S3, Amazon FSx)

Amazon S3 Versioning

S3 Versioning maintains multiple variants of an object in the same bucket. Each version has a unique **version ID**.

Source: Retaining multiple versions of objects with S3 Versioning – Amazon S3

Versioning States

State	Description
Unversioned (default)	No versioning. Objects have no version ID.
Versioning-enabled	All new objects receive a unique version ID. Previous versions are retained.
Versioning-suspended	New objects receive a <code>null</code> version ID. Existing versioned objects are retained.

Once versioning is enabled on a bucket, it cannot be fully disabled — only suspended.

How Deletion Works with Versioning

- **DELETE without version ID:** S3 inserts a **delete marker** as the current version. The object appears deleted but all previous versions are retained.
- **DELETE with version ID:** Permanently deletes that specific version.
- **Restoring a deleted object:** Delete the delete marker to restore the most recent version.

S3 Versioning and Lifecycle Policies

Use S3 Lifecycle policies to manage the cost of versioning:

- **NoncurrentVersionExpiration**: Permanently delete noncurrent versions after a specified number of days.
- **NoncurrentVersionTransition**: Move noncurrent versions to a cheaper storage class (e.g., S3 Glacier Instant Retrieval) after a specified number of days.

S3 Object Lock

S3 Object Lock prevents objects from being deleted or overwritten for a fixed period or indefinitely. It uses a WORM model and supports two retention modes:

Mode	Description
Governance mode	Users with special IAM permissions can override or remove the lock.
Compliance mode	No user, including the root account, can delete or modify the object until the retention period expires.

Object Lock requires versioning to be enabled.

Amazon FSx Versioning and Backups

Amazon FSx for Windows File Server

FSx for Windows File Server supports **shadow copies** (Windows Volume Shadow Copy Service), which create point-in-time snapshots of the file system. Users can restore previous versions of files directly from Windows Explorer without administrator intervention.

- Shadow copies are stored on the file system itself.
- AWS Backup can create daily backups of FSx file systems stored outside the file system for longer retention.

Amazon FSx for NetApp ONTAP

FSx for NetApp ONTAP supports **SnapMirror** (replication to another FSx for ONTAP file system or on-premises ONTAP) and **FlexClone** (instant writable clones of volumes). AWS Backup integration provides additional backup and restore capabilities.

Amazon FSx for OpenZFS

FSx for OpenZFS supports **ZFS snapshots** — point-in-time, space-efficient copies of volumes. Snapshots are stored on the file system and can be used to restore individual files or entire volumes.

Skill 2.3.4 – Follow Disaster Recovery Procedures

Disaster Recovery Strategies

AWS defines four DR strategies, ordered from lowest cost/highest RTO to highest cost/lowest RTO.

Source: Disaster recovery options in the cloud – AWS Whitepaper

Strategy 1: Backup and Restore

Description: Data is backed up to AWS (or within AWS to another Region). In a disaster, infrastructure is redeployed from IaC (CloudFormation, CDK) and data is restored from backups.

Attribute	Value
RTO	Hours
RPO	Hours (depends on backup frequency)
Cost	Lowest
Complexity	Lowest

Key services: AWS Backup, S3 Cross-Region Replication, CloudFormation, AWS CodePipeline.

Best for: Non-critical workloads, cost-sensitive environments, or workloads where hours of downtime are acceptable.

Strategy 2: Pilot Light

Description: A minimal version of the environment is always running in the DR Region. Core data replication is active (e.g., RDS cross-Region read replica, S3 CRR), but compute resources are stopped or minimal. In a disaster, compute is scaled up and traffic is redirected.

Attribute	Value
RTO	Tens of minutes
RPO	Minutes (near-real-time replication)
Cost	Low (minimal running resources)
Complexity	Moderate

Key services: RDS cross-Region read replica, S3 CRR, Route 53 failover routing, CloudFormation, EC2 AMIs.

Best for: Workloads that need faster recovery than backup/restore but can tolerate some downtime.

Strategy 3: Warm Standby

Description: A scaled-down but fully functional version of the production environment runs in the DR Region. In a disaster, the standby is scaled up to full production capacity and traffic is redirected.

Attribute	Value
RTO	Minutes
RPO	Seconds to minutes
Cost	Moderate (running reduced-capacity environment)
Complexity	Moderate to high

Key services: Aurora Global Database, RDS cross-Region read replica, EC2 Auto Scaling, Route 53 failover, ELB.

Best for: Business-critical workloads that require fast recovery with moderate cost.

Strategy 4: Multi-Site Active/Active

Description: The workload runs simultaneously in two or more AWS Regions. Traffic is distributed across all Regions. In a disaster, the failed Region is removed from the rotation and the remaining Regions absorb all traffic.

Attribute	Value
RTO	Near zero (seconds)
RPO	Near zero
Cost	Highest (full duplicate environment)
Complexity	Highest

Key services: Aurora Global Database, DynamoDB Global Tables, Route 53 latency/geolocation routing, Global Accelerator, CloudFront.

Best for: Mission-critical workloads where any downtime is unacceptable (e.g., financial systems, healthcare).

DR Strategy Comparison

Strategy	RTO	RPO	Cost	Use Case
Backup and Restore	Hours	Hours	\$	Dev/test, non-critical
Pilot Light	Tens of minutes	Minutes	\$\$	Internal apps, moderate criticality
Warm Standby	Minutes	Seconds–minutes	\$\$\$	Business-critical apps
Multi-Site Active/Active	Seconds	Seconds	\$\$\$\$	Mission-critical, zero-downtime

AWS Elastic Disaster Recovery (AWS DRS)

AWS Elastic Disaster Recovery (formerly CloudEndure Disaster Recovery) provides continuous block-level replication of on-premises or cloud-based servers to AWS. In a disaster, recovery instances can be launched in minutes.

Key features:

- Continuous replication with sub-second RPO.
- Non-disruptive recovery drills.
- Automated machine conversion (physical/virtual to EC2).
- Supports failback to the original source after recovery.

AWS Resilience Hub

AWS Resilience Hub provides a central place to define, validate, and track the resilience of AWS applications. It:

- Assesses whether an application meets its defined RTO and RPO targets.
- Identifies resiliency gaps and provides recommendations.
- Integrates with AWS Fault Injection Service (FIS) for chaos engineering.

Disaster Recovery Testing

A DR plan is only as good as its last successful test. Best practices:

1. **Define runbooks:** Document step-by-step recovery procedures. Store them in Systems Manager OpsCenter or a wiki.
 2. **Conduct regular DR drills:** Test failover procedures without impacting production (use non-production environments or AWS DRS drill mode).
 3. **Automate failover where possible:** Use Route 53 health checks, RDS automated failover, and Lambda-triggered runbooks to reduce manual steps.
 4. **Measure actual RTO and RPO:** Compare measured values against targets after each drill.
 5. **Use AWS Fault Injection Service (FIS):** Inject controlled failures (e.g., terminate EC2 instances, throttle API calls, inject network latency) to validate resilience.
-

Summary Table: Domain 2 Key Services

Task	Key AWS Services
Compute scaling	EC2 Auto Scaling, Application Auto Scaling, AWS Auto Scaling
Caching	Amazon CloudFront, Amazon ElastiCache (Redis, Memcached)
Database scaling	Amazon RDS (read replicas, storage auto scaling), Amazon Aurora (Auto Scaling, Serverless v2), Amazon DynamoDB (on-demand, auto scaling)
Load balancing	ALB, NLB, GWLB
DNS and health checks	Amazon Route 53 (health checks, failover routing)
Multi-AZ	RDS Multi-AZ, Aurora (multi-AZ by default), ElastiCache Multi-AZ, EFS Standard
Backup	AWS Backup, Amazon DLM, RDS automated backups, EBS snapshots, S3 versioning
Restore	RDS PITR, Aurora PITR, Aurora Backtrack, DynamoDB PITR, S3 version restore
Disaster recovery	AWS Elastic Disaster Recovery, Aurora Global Database, DynamoDB Global Tables, Route 53 failover, AWS Resilience Hub

Key Exam Tips for Domain 2

- **Target tracking scaling** is the recommended and simplest scaling policy for most EC2 Auto Scaling use cases. Know when to use step scaling (fine-grained control) vs. scheduled scaling (predictable patterns).
- **Cooldown periods** apply to simple scaling policies. Target tracking and step scaling use **instance warmup** instead.

- **ElastiCache Redis** supports persistence, replication, and complex data structures. **Memcached** is simpler and supports multi-threading but has no persistence or replication.
- **RDS Multi-AZ** uses synchronous replication and provides automatic failover. **Read replicas** use asynchronous replication and are for read scaling, not automatic failover (though they can be manually promoted).
- **RDS PITR** creates a new DB instance — it does not restore in place. Plan for endpoint changes.
- **S3 Versioning** cannot be fully disabled once enabled — only suspended.
- **S3 Object Lock in Compliance mode** cannot be overridden by anyone, including root. Use Governance mode when you need the ability to override.
- Know the four DR strategies and their RTO/RPO tradeoffs. The exam frequently tests which strategy is appropriate for a given RTO/RPO requirement.
- **AWS Backup Vault Lock** provides WORM protection for backups — useful for compliance scenarios.
- **Aurora Backtrack** rewinds the database in place (no new cluster needed) but only supports MySQL-compatible Aurora and has a maximum backtrack window of 72 hours.

Document generated using AWS official documentation. All sources are linked inline.